



Aula 1 — Os primeiros passos com C

Se você está começando agora, não se preocupe em decorar tudo. Nesta primeira aula, vamos entender a base para começar a escrever programas em C.

Vamos aprender o que é a estrutura básica de um programa, como mostrar informações na tela, como receber informações do utilizador e, principalmente, o que são variáveis e constantes.

A ideia é aprender através de exemplos que realmente poderíamos usar em um programa.

1. A estrutura básica de um programa em C

Vamos começar com o famoso programa:

```
#include <stdio.h>

int main() {

    printf("Ola, mundo!");

    return 0;
}
```

À primeira vista pode parecer muita coisa, mas vamos por partes.

`#include <stdio.h>` permite utilizar algumas funções importantes de entrada e saída, como `printf()` e `scanf()`.

`int main()` é o ponto onde o nosso programa começa a ser executado.

As `{ }` indicam onde começa e termina o conteúdo da função `main`.

`printf()` é utilizado para mostrar alguma informação na tela.

E `return 0;` indica que o programa terminou normalmente.

Por enquanto, não precisamos decorar tudo isso. O mais importante é começar a reconhecer essa estrutura.

2. printf() — mostrando informações na tela

O `printf()` serve basicamente para apresentar alguma coisa ao utilizador.

Por exemplo:

```
#include <stdio.h>

int main() {

    printf("Meu nome e Jose.");
    printf("Estou aprendendo C.");

    return 0;
}
```

Podemos também organizar melhor o texto usando `\n`.

```
#include <stdio.h>

int main() {

    printf("Nome: Jose\n");
    printf("Idade: 18 anos\n");
    printf("Curso: Programacao\n");

    return 0;
}
```

O `\n` significa que queremos passar para uma nova linha.

Então podemos imaginar que o `printf()` é a forma que o nosso programa utiliza para "falar" com o utilizador.

3. Variáveis — guardando informações

Agora chegamos a uma das partes mais importantes da programação: as variáveis.

Uma variável é um espaço na memória que podemos utilizar para guardar uma informação.

Por exemplo, imagine que queremos criar um programa para guardar a idade de uma pessoa.

Podemos fazer:

```
int idade = 18;
```

Aqui criamos uma variável chamada `idade` e guardamos nela o valor `18`.

Podemos depois utilizar essa variável:

```
#include <stdio.h>
```



```
int main() {

    int idade = 18;

    printf("Eu tenho %d anos.", idade);

    return 0;
}
```

O `%d` indica que vamos colocar ali um valor do tipo `int`.

O resultado será:

```
Eu tenho 18 anos.
```

Agora imagine que queremos guardar o nome.

Podemos utilizar:

```
char nome[] = "Jose";
```

E depois:

```
printf("Meu nome e %s.", nome);
```

O `%s` é utilizado para texto.

4. Tipos de variáveis

Nem toda informação é do mesmo tipo.

Por exemplo, uma idade é um número inteiro, enquanto uma altura pode ter casas decimais.

Alguns dos tipos mais utilizados no início são:

```
int        // números inteiros
float      // números com casas decimais
double     // números decimais com maior precisão
char       // um caractere
```

Exemplo:

```
int idade = 18;
```



```
float altura = 1.75;
```

```
char inicial = 'J';
```

Também podemos juntar vários tipos no mesmo programa:

```
#include <stdio.h>

int main() {

    int idade = 18;
    float altura = 1.75;
    char inicial = 'J';

    printf("Idade: %d\n", idade);
    printf("Altura: %.2f\n", altura);
    printf("Inicial: %c\n", inicial);

    return 0;
}
```

Aqui temos:

`%d` → `int`

`%f` → `float`

`%c` → `char`

E `%.2f` significa que queremos mostrar duas casas decimais.

5. scanf() — recebendo informações

Até agora, o nosso programa apenas mostra informações.

Mas um programa de verdade precisa também receber informações.

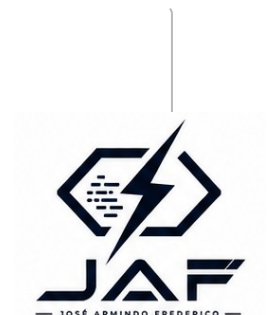
É aí que entra o `scanf()`.

Por exemplo, podemos pedir ao utilizador a idade:

```
#include <stdio.h>
```

```
int main() {
```

```
    int idade;
```



```

    printf("Digite a sua idade: ");
    scanf("%d", &idade);

    printf("Voce tem %d anos.", idade);

    return 0;
}

```

Quando o programa chegar ao `scanf()`, ele vai esperar que o utilizador escreva alguma coisa.

Por exemplo:

```

Digite a sua idade: 18
Voce tem 18 anos.

```

Repare neste detalhe:

```
scanf("%d", &idade);
```

O `&idade` indica onde o programa deve guardar o valor recebido.

Não precisa entrar profundamente nisso agora. Mais à frente vamos entender melhor o que esse `&` significa.

6. Um exemplo mais próximo da realidade

Agora vamos juntar o que aprendemos.

Vamos imaginar que queremos criar um pequeno programa para receber o nome e a idade de uma pessoa.

```

#include <stdio.h>

int main() {

    char nome[50];
    int idade;

    printf("Digite o seu nome: ");
    scanf("%49s", nome);

    printf("Digite a sua idade: ");
    scanf("%d", &idade);

    printf("\n--- DADOS ---\n");
    printf("Nome: %s\n", nome);
    printf("Idade: %d anos\n", idade);
}

```



```
    return 0;  
}
```

Uma possível execução seria:

```
Digite o seu nome: Jose  
Digite a sua idade: 18  
  
--- DADOS ---  
Nome: Jose  
Idade: 18 anos
```

Já conseguimos fazer um pequeno programa que recebe dados de uma pessoa e depois apresenta esses dados.

É simples, mas é exatamente assim que começamos.

7. O que são constantes?

Agora vamos falar de constantes.

Uma constante é um valor que não queremos alterar durante a execução do programa.

Por exemplo, imagine que estamos criando um programa que precisa utilizar o valor de PI.

Podemos escrever:

```
const float PI = 3.14159;
```

O `const` indica que esse valor não deve ser alterado.

Por exemplo:

```
#include <stdio.h>  
  
int main() {  
  
    const float PI = 3.14159;  
  
    printf("Valor de PI: %.5f", PI);  
  
    return 0;  
}
```

Se tentarmos alterar `PI` depois de criada, teremos um erro.



A diferença básica é:

Variável → pode mudar
Constante → não deve mudar

Por exemplo:

```
int idade = 18;  
  
idade = 19;
```

Isso é permitido porque `idade` é uma variável.

Já:

```
const int ANO = 2026;  
  
ANO = 2027;
```

não é permitido porque `ANO` foi definido como constante.

8. Pequeno desafio

Agora quero que você tente fazer sozinho.

Crie um programa que:

1. Pergunte o nome da pessoa.
2. Pergunte a idade.
3. Pergunte a altura.
4. Mostre todas essas informações no final.

A estrutura pode começar assim:

```
#include <stdio.h>  
  
int main() {  
  
    char nome[50];  
    int idade;  
    float altura;  
  
    // seu código aqui
```



```
    return 0;  
}
```

Não se preocupe se não conseguir de primeira. Tente montar o programa usando `printf()`, `scanf()` e as variáveis que aprendemos nesta aula.

O que aprendemos nesta aula

Nesta primeira aula, conhecemos:

- A estrutura básica de um programa em C;
- `printf()` para mostrar informações;
- `scanf()` para receber informações;
- O que são variáveis;
- Os principais tipos de variáveis;
- `int`, `float`, `double` e `char`;
- Como utilizar variáveis dentro do `printf()`;
- O que são constantes;
- A diferença entre variável e constante;
- Como criar um pequeno programa utilizando tudo isso.

E esse é apenas o começo.

Na próxima aula, podemos começar a trabalhar com operadores e fazer o programa deixar de apenas guardar informações para também realizar cálculos e tomar decisões.

